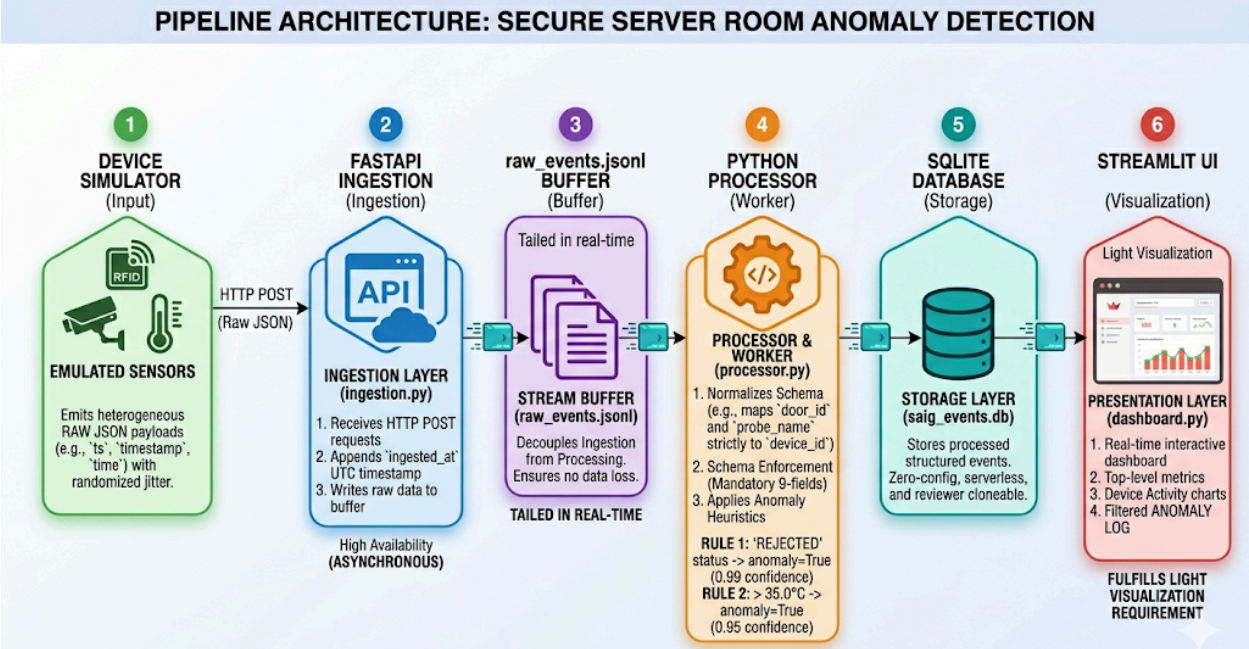
Architecture & Data Flow

1. Architecture Diagram Instructions



Architecture Note

1. Data Sources The system relies on a custom Python simulation layer (`simulator.py`) that acts as three distinct physical devices within a "Secure Server Room":

- **RFID Badge Reader (Access Control):** Emits access events (Granted/Rejected).
- **Camera Motion Sensor (Binary Trigger):** Emits motion detection events.
- **Temperature Sensor (Environmental):** Emits continuous ambient temperature readings in Celsius. *To simulate real-world hardware inconsistencies, these sources intentionally emit heterogeneous JSON payloads (e.g., using different keys like `ts`, `timestamp`, and `time`) and utilize randomized jitter to mimic realistic network latency.*

2. Ingestion Flow Data is ingested via a lightweight, asynchronous HTTP REST API built with FastAPI (`ingestion.py`).

- **Protocol:** Devices send payloads via HTTP POST requests to the `/ingest` endpoint.
- **Timestamping:** The API immediately appends an `ingested_at` UTC timestamp to track system latency.
- **Decoupling Strategy:** To ensure high availability, the API does not pass data directly to the processor. Instead, it appends the raw JSON payload to a local stream buffer

(`raw_events.jsonl`). This decoupling ensures that if the processing layer goes down, no incoming sensor data is lost.

3. Processing / Transformation A dedicated worker script (`processor.py`) continuously tails the `.jsonl` stream buffer in real-time.

- **Normalization:** It handles the messy, heterogeneous data by mapping inconsistent keys (e.g., mapping `door_id` or `probe_name` strictly to `device_id`).
- **Schema Enforcement:** It structures the cleaned data into the exact 9-field mandatory schema required by the business (including `event_id`, `device_type`, and `raw_source_reference`).

4. Storage (if any) Processed and structured events are stored in a local **SQLite database** (`saig_events.db`).

- **Reasoning:** SQLite was chosen for its zero-configuration, serverless nature. This satisfies the requirement for a fully functional local pipeline while ensuring reviewers can easily clone and run the system without installing external database infrastructure like PostgreSQL.

5. Analysis Layer It flags two distinct anomalies:

- **Rule 1 (Unauthorized Access):** If an access control payload returns a status of "REJECTED", the system updates the `event_type` to `unauthorized_access_attempt`, sets the `anomaly_flag` to `True`, and assigns a 0.99 `confidence_score`.
- **Rule 2 (Cooling Failure):** If an environmental sensor reads > 35.0°C, it is flagged as a `critical_temperature_spike` with a 0.95 `confidence_score`.

6. Presentation Layer The output is visualized using a **Streamlit web application** (`dashboard.py`).

- This fulfills the "light visualization" requirement by executing SQL queries against the SQLite database and rendering the data into a real-time, interactive dashboard.
- It provides top-level metrics, a bar chart of device activity, a filtered view of critical anomalies, and a full structured event log conforming to the mandatory output schema.

7. Known Limitations

- **Scalability:** SQLite and a `.jsonl` file buffer are perfect for local prototyping and fulfill the assignment's. However, in a production environment processing millions of events per second, these would bottleneck. They would need to be replaced by a distributed message broker (like Kafka or AWS Kinesis) and a scalable data warehouse (like Snowflake or BigQuery).
- **Security:** The current FastAPI ingestion endpoint does not require authentication (e.g.,

API keys or OAuth). In production, the endpoint would require mutual TLS or token-based authentication to prevent bad actors from spoofing sensor data.

- **Rigid Heuristics:** The anomaly detection uses hardcoded thresholds (e.g., exactly > 35°C). While highly defensible for this prototype, a mature system would eventually incorporate statistical anomaly detection to catch "unknown unknown" anomalies dynamically.